

CO5 - A1 ASSESSMENT

NIIVEDHITHA G

192521231

CSA0313

DATA STRUCTURE

- 1) Social Network Graph - Connected components & shortest connection.

Pseudocode:

ALGORITHM Social Network (G)

visited $\leftarrow$ empty set

FOR each user u in G

IF u not in visited

BFS (G, u)

PRINT "New Connected Component"

END IF

END FOR

shortest $\leftarrow$ DIJKSTRA (G, SOURCE)

END ALGORITHM.

- 2) Task Dependencies - Topological sorting.

Pseudocode:

ALGORITHM Task scheduling (G)

Calculate indegree of every task

Queue $\leftarrow$ all tasks with indegree = 0

Order $\leftarrow$ empty list.

WHILE Queue is not empty

$u \leftarrow$ remove task from Queue

Add u to Order.

FOR each v adjacent to u

indegree [v] $\leftarrow$ indegree [v] - 1

IF indegree [v] = 0


```

    Add v to Queue
  END IF
END FOR
END WHILE

If size (order)  $\neq$  number of tasks
  PRINT "Cycle exists - no valid order"
ELSE
  PRINT Order
END IF
END ALGORITHM.

```

3) Task scheduling system
Pseudocode:

```

ALGORITHM Scheduling Tasks (Tasks, Dependencies)
  Create directed graph G
  FOR each dependency (A  $\rightarrow$  B)
    Add edge A  $\rightarrow$  B
  END FOR.

  Order  $\rightarrow$  TOPOLOGICAL - SORT(G)

  IF order is empty
    PRINT "Cyclic dependency exists"
  ELSE
    PRINT "Valid Execution Order:", Order
  END IF

  END ALGORITHM.

```


4) DFS with cycle Detection

Pseudo code:

ALGORITHM DFS (G, v)

visited [v] $\leftarrow$ TRUE

recursionstack [v] $\rightarrow$ TRUE

FOR each v adjacent to u

IF visited [v] = FALSE

IF DFS (G, v) = TRUE

RETURN TRUE

END IF

ELSE IF recursion stack [v] = TRUE

RETURN TRUE

END IF

END FOR

recursion stack [u] $\leftarrow$ FALSE

RETURN FALSE

END ALGORITHM

5) Delivery system - Shortest Route

ALGORITHM DeliveryRoute ($G, source$)

FOR each vertex v

distance [v] $\leftarrow \infty$

END FOR

distance [$source$] $\leftarrow 0$

Priority Queue $\leftarrow source$

WHILE Priority Queue is not empty

$u \leftarrow$ vertex with minimum distance


```

For each edge  $(u, v)$  with weight  $w$ 
  new Distance  $\leftarrow$  distance  $[u] + w$ 
  If new Distance  $<$  distance  $[v]$ 
    distance  $[v] \leftarrow$  new distance
  Insert  $v$  into Priority Queue
END IF
END FOR
END WHILE
PRINT distance
END ALGORITHM.

```

6) Kruskal's Algorithm - Minimum spanning Tree

```

ALGORITHM Kruskal (G)
  MST  $\leftarrow$  empty set
  sort all edges by increasing weight
  Create separate set for every vertex
  For each edge  $(u, v, w)$  in sorted edges
    IF FIND  $(u) \neq$  FIND  $(v)$ 
      Add edge  $(u, v)$  to MST
      UNION  $(u, v)$ 
    END IF
  If number of MST edges  $= v - 1$ 
    PRINT MST
  ELSE
    PRINT "Graph is not connected"
  END IF
END ALGORITHM.

```


7) Floyd - Warshall - All - Pairs Shortest Path.

ALGORITHM Floyd Warshall (0, v)

FOR $k \leftarrow 1$ TO v

FOR $i \leftarrow 1$ TO v

FOR $j \leftarrow 1$ TO v

$D[i][j] \leftarrow \min(D[i][j], (D[i][k] + D[k][j]))$

END FOR

END FOR

END FOR

FOR $i \leftarrow 1$ TO v

IF $D[i][i] < 0$

PRINT "Negative Cycle Detected"

RETURN

END IF

END FOR

PRINT D

END ALGORITHM.

8) Modified Dijkstra - Avoid Certain Nodes.

ALGORITHM Floyd Warshall (0, v)

FOR $k = 1$ TO v

FOR $i = 1$ TO v

FOR $j = 1$ TO v

$D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$

END FOR

END FOR

END FOR

FOR $i = 1$ TO v

If $DEIT[i] < 0$

Print "Negative Cycle"

END IF

END FOR

END ALGORITHM.

9) Priority Topological sort

ALGORITHM topological sort (A)

Calculate indegree of all nodes

Add all nodes with indegree 0 to priority queue.

WHILE Priority Queue is not empty

$v =$ Remove highest - Priority node

PRINT v

FOR each v adjacent to v

$\text{indegree}[v] = \text{indegree}[v] - 1$

IF $\text{indegree}[v] = 0$

Add v to Priority Queue.

END IF

END FOR

END WHILE

END ALGORITHM.

10) Modified Dijkstra - Avoid Nodes.

ALGORITHM Modified Dijkstra (u, s, o, Avoid)

Set distance $[s] = 0$

Set all others distances = ∞

WHILE nodes remain

$v =$ node with minimum distance

skip v

END IF

for each neighbor v .

Update shortest distance to v

END FOR

END WHILE

PRINT shortest path from s to D

END ALGORITHM.